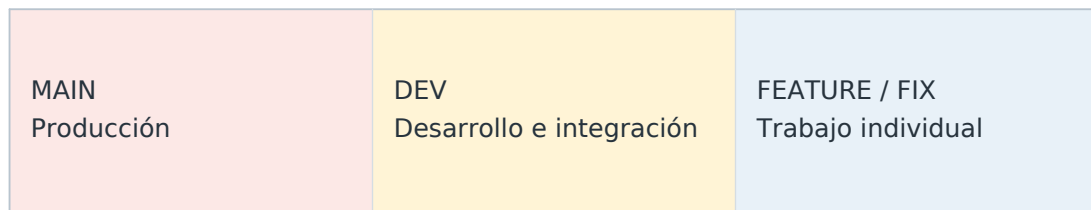


GUÍA DE TRABAJO CON GIT

Flujo de trabajo del equipo y gestión del repositorio



Regla central

feature/* → Pull Request → dev → pruebas → Pull Request → main → PRODUCCIÓN

Documento de referencia para el equipo

1. Objetivo

Este documento define la forma en que todo el equipo debe trabajar con Git y con el repositorio del proyecto.

El objetivo es:

- Mantener el código organizado.
- Evitar que varios integrantes sobrescriban el trabajo de otros.
- Mantener `main` estable y listo para producción.
- Utilizar `dev` como rama de integración y desarrollo.
- Facilitar la revisión de cambios mediante Pull Requests.
- Mantener un historial de commits claro y entendible.

2. Estructura de ramas

El repositorio tendrá como ramas principales:

```
main
■■■ Producción

dev
■■■ Desarrollo / integración
```

□ `main` --- Producción

La rama `main` representa ****exclusivamente el código que está en producción****.

Reglas

- □ No se trabaja directamente sobre `main`.
- □ No se deben hacer commits directamente sobre `main`.
- □ No se deben subir cambios experimentales.
- □ No se debe utilizar `main` para desarrollar funcionalidades.
- □ Los cambios llegan a `main` únicamente después de haber sido desarrollados y probados.
- □ La incorporación de cambios a `main` debe hacerse mediante Pull Request desde `dev`.
- □ Antes de pasar cambios a `main`, el equipo debe verificar que la versión está lista para producción.

Regla principal: si algo todavía está en desarrollo, no pertenece a `main`.

□ dev --- Desarrollo

La rama `dev` es la rama donde se integra el trabajo que está siendo desarrollado por el equipo.

Reglas

- □ No se debe trabajar directamente sobre `dev` para desarrollar una funcionalidad.
- □ No se deben subir cambios incompletos directamente a `dev`.
- □ Cada integrante debe crear una rama propia para su tarea.
- □ Las ramas de trabajo se crean a partir de `dev`.
- □ Cuando una tarea está terminada, se crea un Pull Request hacia `dev`.
- □ Los cambios deben revisarse antes de integrarse.

Regla principal: `dev` es la rama de integración, no la rama personal de ningún integrante.

3. Flujo general de trabajo

El flujo recomendado es:



En términos simples:

```
dev
├─
├─ feature/login
├─ feature/usuarios
├─ fix/error-login
├─ refactor/database
├─
├─
├─ dev
├─
├─ Cuando está listo para producción
├─
├─
└─ main
```

4. Configuración inicial --- Solo la primera vez

Cada integrante debe clonar el repositorio.

```
git clone URL_DEL_REPOSITORIO
```

Entrar al proyecto:

```
cd NOMBRE_DEL_PROYECTO
```

Verificar las ramas:

```
git branch
```

También se pueden consultar las ramas remotas:

```
git branch -a
```

5. Configurar usuario de Git

Cada integrante debe tener configurado su nombre y correo.

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@example.com"
```

Para verificar:

```
git config --global user.name
git config --global user.email
```

Se recomienda utilizar el mismo correo asociado a la cuenta de GitHub/GitLab utilizada para el proyecto.

6. Antes de comenzar cualquier tarea

Nunca se debe empezar una tarea con una rama desactualizada.

Primero:

```
git switch dev
```

Después:

```
git pull origin dev
```

Ahora la rama `dev` local estará actualizada.

7. Crear una rama para trabajar

Cada tarea debe realizarse en una rama independiente.

La rama debe crearse desde `dev`.

Ejemplo:

```
git switch dev
git pull origin dev
git switch -c feature/login
```

Esto crea:

```
dev
■ ■ ■ feature/login
```

A partir de ese momento, el desarrollador trabaja únicamente en:

```
feature/login
```

8. Convención para nombrar ramas

Los nombres de las ramas deben ser claros y descriptivos.

Se recomienda utilizar:

Tipo Uso Ejemplo

`feature/` Nueva funcionalidad `feature/login`

`fix/` Corrección de errores `fix/error-login`

`refactor/` Reestructuración del código `refactor/database`

`docs/` Documentación `docs/readme`

`test/` Pruebas `test/users`

`chore/` Tareas técnicas/mantenimiento `chore/docker`

Ejemplos

```
feature/registro-usuarios
feature/autenticacion
feature/gestion-apuestas

fix/error-validacion
fix/error-conexion-db

refactor/modelos-database

docs/guia-git

test/login

chore/docker
```

Evitar

```
prueba
cambio
cosas
mi-rama
rama-john
final
final2
final-ahora-si
```

El nombre debe indicar qué se está haciendo, no quién lo está haciendo.

9. Trabajar en la rama

Una vez creada la rama:

```
git switch feature/nombre-de-la-tarea
```

Se realizan los cambios necesarios.

Durante el desarrollo es recomendable comprobar periódicamente:

```
git status
```

Este comando permite saber:

- Qué archivos fueron modificados.
- Qué archivos están preparados para commit.
- Si existen archivos sin seguimiento.
- En qué rama estamos trabajando.

10. Guardar cambios con `git add`

Cuando una parte del trabajo está lista:

```
git add .
```

También se puede agregar un archivo específico:

```
git add archivo.py
```

Antes de hacer commit se recomienda revisar:

```
git status
```

11. Crear commits

Los commits deben representar cambios concretos y entendibles.

Ejemplo:

```
git commit -m "feat: agregar autenticación de usuarios"
```

Otros ejemplos:

```
git commit -m "fix: corregir validación del correo"
git commit -m "docs: actualizar guía de instalación"
git commit -m "refactor: reorganizar servicios de usuarios"
git commit -m "test: agregar pruebas para login"
```

Convención recomendada

Utilizar:

```
tipo: descripción
```

Tipos principales:

- **feat** → nueva funcionalidad.
- **fix** → corrección de errores.
- **docs** → documentación.
- **refactor** → modificación interna sin cambiar la funcionalidad.
- **test** → pruebas.
- **chore** → mantenimiento o configuración.

Buen commit

```
feat: agregar endpoint para crear apuestas
```

Mal commit

```
cambios
```

```
arreglos
```

```
cosas varias
```

```
final
```

12. ¿Cada cuánto hacer commits?

No es necesario esperar a terminar toda una funcionalidad para hacer un commit.

Se recomienda hacer commits cuando se completa una unidad lógica de trabajo.

Por ejemplo:

```
feat: crear modelo de usuario  
feat: agregar endpoint de usuarios  
test: agregar pruebas de usuarios  
fix: corregir validación de usuario
```

Evitar un único commit gigante como:

```
feat: hacer todo el módulo de usuarios
```

13. Subir la rama al repositorio remoto

Después de realizar los commits:

```
git push -u origin feature/nombre-de-la-tarea
```

En los siguientes [push](#) de esa misma rama normalmente bastará con:

```
git push
```

14. Pull Request hacia dev

Cuando la tarea esté terminada:

```
feature/nombre-de-la-tarea  
  ■  
  ■ Pull Request  
  ▼  
dev
```

El Pull Request debe explicar:

Qué se hizo

Ejemplo:

```
Se agregó el sistema de autenticación de usuarios.
```

Qué problema resuelve

```
Permite que los usuarios puedan iniciar sesión utilizando correo y contraseña.
```

Qué se debe probar

- Registro de usuario.
- Inicio de sesión correcto.
- Contraseña incorrecta.
- Usuario inexistente.

Antes de crear el Pull Request

Verificar:

- ☐ El código funciona.
- ☐ Se realizaron las pruebas necesarias.
- ☐ No hay archivos innecesarios.
- ☐ No hay contraseñas ni secretos en el código.
- ☐ El código cumple con las convenciones del proyecto.
- ☐ La rama está actualizada con `dev`.
- ☐ Los commits tienen mensajes claros.

15. Mantener la rama actualizada con dev

Mientras se está trabajando, otros integrantes pueden incorporar cambios a `dev`.

Por eso, antes de finalizar una tarea, se recomienda actualizar la rama.

Primero:

```
git switch dev
git pull origin dev
```

Luego regresar a nuestra rama:

```
git switch feature/nombre-de-la-tarea
```

Después integrar los cambios de `dev`:

```
git merge dev
```

Si aparecen conflictos, deben resolverse antes de continuar.

Finalmente:

```
git push
```

16. ¿Qué hacer si aparece un conflicto?

Un conflicto ocurre cuando Git detecta que dos ramas modificaron la misma parte de un archivo de manera incompatible.

Git mostrará algo parecido a:

```
<<<<<< HEAD
código de nuestra rama
=====
código proveniente de dev
>>>>>> dev
```

Se debe:

1. Abrir el archivo.
2. Revisar las dos versiones.
3. Decidir qué código debe permanecer.
4. Eliminar los marcadores:

```
<<<<<<
=====
>>>>>>
```

5. Guardar el archivo.
6. Marcarlo como resuelto:

```
git add archivo
```

7. Completar el merge:

```
git commit
```

8. Subir los cambios:

```
git push
```

Si un conflicto no está claro, no se debe elegir una versión al azar.

Se debe consultar al responsable de la parte del código involucrada.

17. Integración de cambios a dev

El flujo será:

```
Desarrollador
  ▼
feature/nueva-funcionalidad
  ▼
Pull Request
  ▼
Revisión
  ▼
dev
```

La rama `dev` debe contener código que pueda integrarse y probarse.

No se deben utilizar `dev` como un lugar para almacenar código roto o experimentos personales.

18. Paso de dev a main

`main` es producción.

Por lo tanto:

```
dev
  ▼
Pull Request
  ▼
main
```

Este proceso debe realizarse únicamente cuando una versión esté lista para producción.

Antes de realizar el Pull Request:

- Se deben haber probado los cambios.
- Las funcionalidades principales deben funcionar.
- No deben existir errores conocidos que impidan utilizar la aplicación.
- Los cambios deben estar integrados correctamente en `dev`.
- El equipo debe revisar qué cambios entrarán en producción.

19. Regla de oro para main

❗ **NUNCA hacer esto:**

```
git switch main
# modificar archivos
git add .
git commit
git push
```

El flujo correcto es:

```
feature/*  
  ↓  
Pull Request  
  ↓  
dev  
  ↓  
Pruebas  
  ↓  
Pull Request  
  ↓  
main  
  ↓  
Producción
```

20. Comandos que se utilizarán con mayor frecuencia

Ver estado

```
git status
```

Ver ramas

```
git branch
```

Cambiar de rama

```
git switch nombre-rama
```

Actualizar dev

```
git switch dev  
git pull origin dev
```

Crear rama

```
git switch -c feature/nombre
```

Preparar cambios

```
git add .
```

Crear commit

```
git commit -m "tipo: descripción"
```

Subir cambios

```
git push
```

Descargar cambios

```
git pull
```

Ver historial

```
git log --oneline
```

21. Flujo completo de una tarea

Supongamos que la tarea es:

Crear el endpoint para registrar usuarios.

Paso 1 --- Actualizar dev

```
git switch dev  
git pull origin dev
```

Paso 2 --- Crear rama

```
git switch -c feature/registro-usuarios
```

Paso 3 --- Desarrollar

Se realiza el trabajo necesario.

Paso 4 --- Revisar cambios

```
git status
```

Paso 5 --- Crear commit

```
git add .  
git commit -m "feat: agregar registro de usuarios"
```

Paso 6 --- Subir rama

```
git push -u origin feature/registro-usuarios
```

Paso 7 --- Crear Pull Request

```
feature/registro-usuarios  
↓  
dev
```

Paso 8 --- Revisar y corregir

Si el Pull Request requiere cambios:

```
# realizar cambios  
  
git add .  
git commit -m "fix: corregir validación de registro"  
git push
```

El Pull Request se actualizará automáticamente.

Paso 9 --- Integrar en dev

Una vez aprobado, se hace merge hacia `dev`.

Paso 10 --- Probar

El equipo prueba la integración en `dev`.

Paso 11 --- Preparar producción

Cuando la versión esté lista:

```
dev  
↓  
Pull Request  
↓  
main
```

Paso 12 --- Producción

Una vez aprobado el Pull Request hacia `main`, los cambios pasan a producción.

22. ¿Qué pasa después de hacer merge?

Una vez que la rama fue integrada en `dev`, la rama de trabajo puede eliminarse.

Por ejemplo:

```
feature/registro-usuarios
```

Ya no es necesaria después del merge.

En el repositorio remoto:

```
git push origin --delete feature/registro-usuarios
```

En el equipo local:

```
git branch -d feature/registro-usuarios
```

No eliminar una rama si todavía contiene trabajo que no haya sido integrado.

23. Archivos que NO deben subirse al repositorio

Nunca subir información sensible.

Por ejemplo:

```
.env  
.env.*  
*.key  
*.pem  
secretos  
contraseñas  
tokens  
credenciales
```

Estos archivos deben estar incluidos en `.gitignore` cuando corresponda.

Ejemplo:

```
.env  
.env.*  
__pycache__/  
*.pyc  
.venv/  
venv/
```

❏ Importante

Nunca escribir contraseñas directamente en el código:

```
PASSWORD = "123456"
```

Ni subir:

```
DATABASE_URL=postgresql://usuario:contraseña@servidor/basedatos
```

Las credenciales deben manejarse mediante variables de entorno o el mecanismo de secretos definido por el proyecto.

24. Antes de hacer push

Cada desarrollador debe revisar:

```
git status
```

Y preguntarse:

- ¿Estoy en mi rama?
- ¿Estoy subiendo únicamente los archivos relacionados con mi tarea?
- ¿Hay archivos sensibles?
- ¿El proyecto funciona?
- ¿El commit tiene un mensaje claro?
- ¿Necesito actualizar mi rama con `dev`?

25. Antes de hacer Pull Request

Checklist:

- [] Estoy trabajando en la rama correcta.
- [] La rama fue creada desde `dev`.
- [] Mi código funciona.
- [] Las pruebas necesarias fueron realizadas.
- [] No incluí credenciales.
- [] No incluí archivos temporales.
- [] Mi rama está actualizada con `dev`.
- [] Los commits son claros.
- [] El Pull Request explica qué se modificó.
- [] El Pull Request indica cómo probar los cambios.

26. Reglas generales del equipo

Regla 1

Nunca trabajar directamente sobre `main`.

Regla 2

No desarrollar directamente sobre `dev`.

Regla 3

Cada tarea debe tener su propia rama.

Regla 4

Las ramas de trabajo siempre parten de `dev`.

Regla 5

Los cambios llegan a `dev` mediante Pull Request.

Regla 6

Los cambios llegan a `main` mediante Pull Request desde `dev`.

Regla 7

`main` representa producción.

Regla 8

No subir credenciales, contraseñas, tokens ni archivos `.env`.

Regla 9

Los commits deben explicar qué cambio se realizó.

Regla 10

****Si existe un conflicto que no se entiende, consultar antes de resolverlo.****

27. Resumen visual



Flujo que todo integrante debe recordar

```
# 1. Actualizar desarrollo
git switch dev
git pull origin dev

# 2. Crear rama
git switch -c feature/mi-tarea

# 3. Trabajar
# ... realizar cambios ...

# 4. Guardar cambios
git add .
git commit -m "feat: describir cambio"

# 5. Subir rama
git push -u origin feature/mi-tarea

# 6. Crear Pull Request
# feature/mi-tarea → dev

# 7. Después del merge, probar en dev

# 8. Cuando la versión esté lista:
# dev → Pull Request → main
```

28. Filosofía del flujo

La idea fundamental es separar claramente desarrollo de producción:



`dev` es donde se integra y prueba el trabajo del equipo.

`main` es exclusivamente producción.

Cada integrante trabaja en su propia rama, integra mediante Pull Request y mantiene el historial del proyecto limpio y entendible.